QuantRule — Architecture Characteristics

QuantRule

2026-11-12

Contents

1 QuantRule — Architecture Characteristics **1**
1.1 1. Derivation . . . 1
1.2 2. Driving characteristics . . . 1
1.3 3. Supporting characteristics . . . 2
1.4 4. Deliberately not optimized . . . 2
1.5 5. Style, quantum, and planes . . . 3
1.6 6. The determinism claim . . . 3
1.7 7. Trade-offs . . . 3

1 QuantRule — Architecture Characteristics

1.1 1. Derivation

Mark Richards and Neal Ford define an architecture characteristic as a non-domain design consideration that shapes structure and is critical to success. Each one below is tied to a sentence in the kata, carries a fitness function, and states the evidence that exists today.

Demand from the kata	Characteristic
“the same evidence, every time”	Reproducibility (determinism)
“no look-ahead”; events appear only when knowable	Temporal integrity (causality)
“inspect, reproduce, and defend”	Auditability (evidence traceability)
“walk away”; the run outlives the client	Detachability
“work can exceed one machine”	Scalability
“pure filters”; one-way dependencies	Modularity
“one command”; one port	Deployability
“every reading and event has evidence”	Testability (enabling)

1.2 2. Driving characteristics

Characteristic	Good means	Fitness function	Evidence
Reproducibility	Same inputs plus the same image digest produce byte-identical data artifacts	Canonical digest equality; determinism tests; golden fixtures	Strong for the engines; the distributed claim is bounded (see §6)
Temporal integrity	No value at bar t depends on a later bar; events appear only at or after their availability bar	Causality and prefix tests	Strong
Auditability	Every result names data version, config hash, model version, image digest, canonical digest	Provenance fields; bundle-completeness test	Designed; implemented by `DIST-034`

1.3 3. Supporting characteristics

Characteristic	Good means	Fitness function
Detachability	A run completes after the client disconnects	Submit, close, reopen, observe `succeeded`
Scalability	Model and backtest workers scale with no change in output	One worker and four workers produce the same digest
Modularity	Six services, one responsibility each; engines untouched	Service catalog; boundary guard tests
Deployability	One command starts the stack; one port is exposed	`make up`; health check
Research isolation	Hindsight research cannot reach the signal path	Guard: no run artifact write, no strategy read
Testability	Claims are provable at the tier that exercises them	Unit, composed, and port-parity tiers

Scalability is a supporting characteristic, not a driver. It exists because the work is batch and CPU-bound, not because there are many users.

1.4 4. Deliberately not optimized

Characteristic	Decision	Consequence
Availability	A run is a job; a failure is a state	No failover, no uptime target
Multi-tenancy	One principal; profiles are organisational	A shared host is trusted
Low latency	Batch; a run takes as long as it takes	Simplicity wins over speed
Security depth	Host and network boundary, not application auth	Remote exposure needs a network gate

Characteristic	Decision	Consequence
Interoperability	File and JSON contracts only	No third-party API

1.5 5. Style, quantum, and planes

Style. Service-Based Architecture with pipes-and-filters inside each service, and an event-driven control plane.

Quantum. Six deployables, one database instance with schema-per-service, one object store, one image. The set is one quantum for the operator; each service is a separate failure and scaling unit.

Plane	Style	Determinism	Queue
Data	Synchronous, fixed order	Required	no
Control	Asynchronous	Not required	yes
Research	Synchronous, interactive	Bounded	no

```
browser / shell / desktop shell
            | HTTP
            v
        gateway -- postgres
            |           ^
            |           +-- evidence -- minio
            v
        rabbitmq -- backtest -- minio
                        |   +-- model
                        +------ data
        simulation -- (research plane, isolated)
```

Why this is the least worst. A monolith cannot detach a run from the client or use a second machine. Microservices add sagas, a mesh, and version skew for no additional driver. Event-driven everywhere makes byte-identical output depend on delivery order. Space-based and SOA/ESB have no load profile or integration estate to serve.

1.6 6. The determinism claim

Compared by the **canonical digest**: SHA-256 over the sorted list of `(path, sha256)` for `results.json`, `*.csv`, and `*.ndjson`, excluding `provenance.json`, logs, and any file that records where or when the run happened.

Enforced, not sampled: no transcendental or fused-multiply-add operation in the signal path, a pinned target triple, fixed accumulation order, and float-roundtrip serialization. Scale invariance is tested locally. End-to-end confirmation across two CPUs belongs to reviewer verification, and no document claims otherwise.

The research plane records an **inputs digest** and no output digest, because human marks are judgment.

1.7 7. Trade-offs

Trade-off	Decision	Consequence
Reproducibility vs distribution	Distribution accepted; determinism defended by invariants	The claim must be precise
Simplicity vs distribution	Simplicity conceded	Docker becomes mandatory for local use
Scalability vs determinism	Shard only where the merge is order-independent	Backtest stays sequential per run
Auditability vs storage	Duplicate copies retained and hashed	Storage is cheap
Deployability vs native UX	Browser is primary; desktop is a shell	No native file dialogs
Organisation vs isolation	Profiles scope records	They are not a security boundary

Priority if forced to shed one: shed Evolvability first. Never shed Reproducibility, Temporal integrity, Auditability, or Detachability.